MAST387_Assignment 3_TessaSkelly_40265993 (1)

March 17, 2026

1 MAST387 Assignment 3

1.1 Tessa Skelly (40265993)

1.2 A little instruction.

Please store your solutions for the first 12 questions in a dictionary called `results` and ensuring your notebook runs from the start to completion without any errors.

You can check that you have answers for all questions in your `results` dictionary with this code:

```
assert set(results.keys()) == {
    "ex2_birth_weight_lower",
    "ex3_smoking_coefficient",
    "ex4_smoking_coefficient",
    "ex6_compare_white_black",
    "ex6_compare_white_black_tstat",
    "ex7_predictions",
    "ex8_smoking_coefficient",
    "ex10_num_high_drug_arrest",
    "ex11_diff_in_diff",
    "ex12_diff_in_diff_weighted",
}
```

1.3 Question 1: Maternal Smoking and Birth Weight

These days, it is widely understood that mothers who smoke during pregnancy risk exposing their babies to many health problems. This was not common knowledge fifty years ago. One of the first studies that addressed the issue of pregnancy and smoking was the Child Health and Development Studies, a comprehensive study of all babies born between 1960 and 1967 at the Kaiser Foundation Hospital in Oakland, CA. The original reference for the study is Yerushalmy (1964, American Journal of Obstetrics and Gynecology, pp. 505-518). The data and a summary of the study are in Nolan and Speed (2000, Stat Labs, Chapter 10) and can be found at the book's website.

There were about 15,000 families in the study. We will only analyze a subset of the data, in particular 1236 male single births where the baby lived at least 28 days. The researchers interviewed mothers early in their pregnancy to collect information on socioeconomic and demographic characteristics, including an indicator of whether the mother smoked during pregnancy. The variables in the dataset are described in the code book here. In this question, we will attempt to use this data to answer the following questions:

1. Do mothers who smoke tend to give birth to babies with lower weights than mothers who do not smoke?
2. What is a likely range for the difference in birth weights for smokers and non-smokers?
3. Is there any evidence that the association between smoking and birth weight differs by mother's race? If so, characterize those differences.
4. Are there other interesting associations with birth weight that are worth mentioning?

[26]:
```
import pandas as pd
import seaborn.objects as so
import seaborn_objects_recipes as sor
import statsmodels.formula.api as smf
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.linear_model import LinearRegression, Ridge, Lasso
import matplotlib.pyplot as plt
import warnings

# results dictionary
results = {}
```

1.3.1 Question 1-1

Load the data "smoking.csv", which includes information on both biometrics of infants at birth, and information on mothers (variables prefixed with the letter "m"), from Moodle. This time I give you cleaned data to save your time, so please enjoy it.

[27]:
```
smoking = pd.read_csv("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast␣
↪387\\assignment 3\\smoking.csv")
```

1.3.2 Question 1-2

Start by plotting the relationship between infant weight at birth and gestation (length of pregnancy (in days) at time of birth) for both mothers who smoke and those who do not in a single plot. Include regression lines for both populations (again, all on one plot).

Limit attention in the plot (and in subsequent questions) to children born after *more than* 225 days of gestation (there aren't really any observations for parents who smoke for less than that, so we don't get common support).

Does it seem like birthweights tend to be lower for the children of parents who smoke at a given gestational period? Store your answer as either `"yes"` or `"no"` in `ex2_birth_weight_lower`.

NOTE: This dataset has at least one column name that cause issues with Python (and seaborn. and statsmodels).

[28]:
```
smoking.info()
```

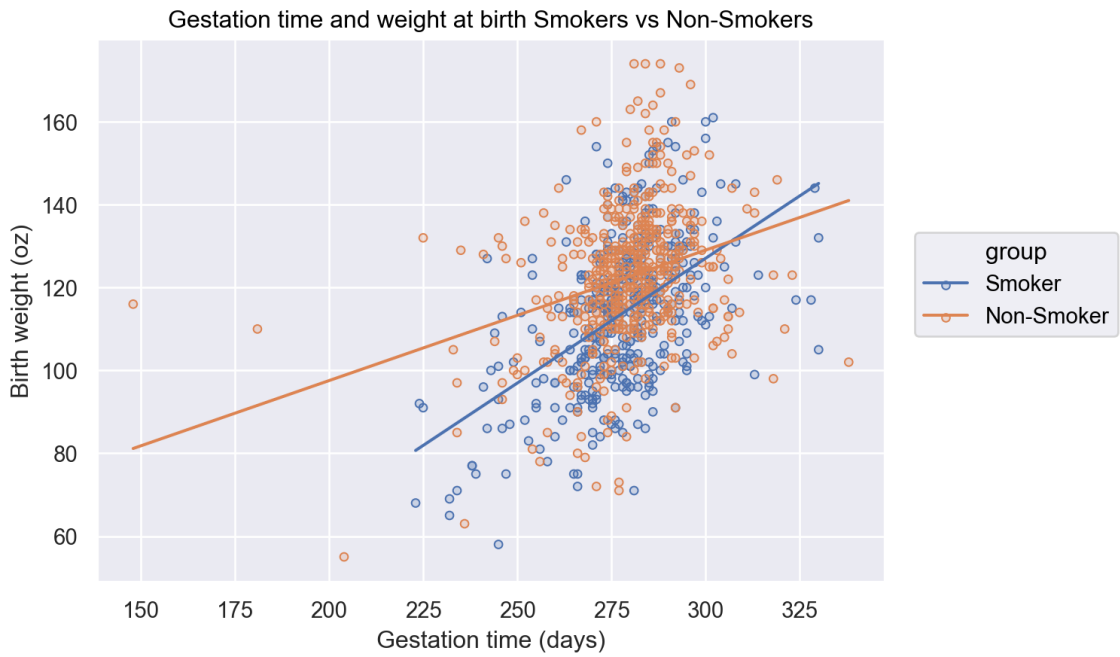
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 869 entries, 0 to 868
```

```
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id          869 non-null    int64
 1   date        869 non-null    int64
 2   gestation   869 non-null    int64
 3   bwt.oz      869 non-null    int64
 4   parity      869 non-null    int64
 5   mrace       869 non-null    int64
 6   mage        869 non-null    int64
 7   med         869 non-null    int64
 8   mht         869 non-null    int64
 9   mpregwt     869 non-null    int64
 10  inc         869 non-null    int64
 11  smoke       869 non-null    int64
dtypes: int64(12)
memory usage: 81.6 KB
```

[29]:
```
smokers = smoking[smoking["smoke"]==1]
non_smokers = smoking[smoking["smoke"]==0]
```

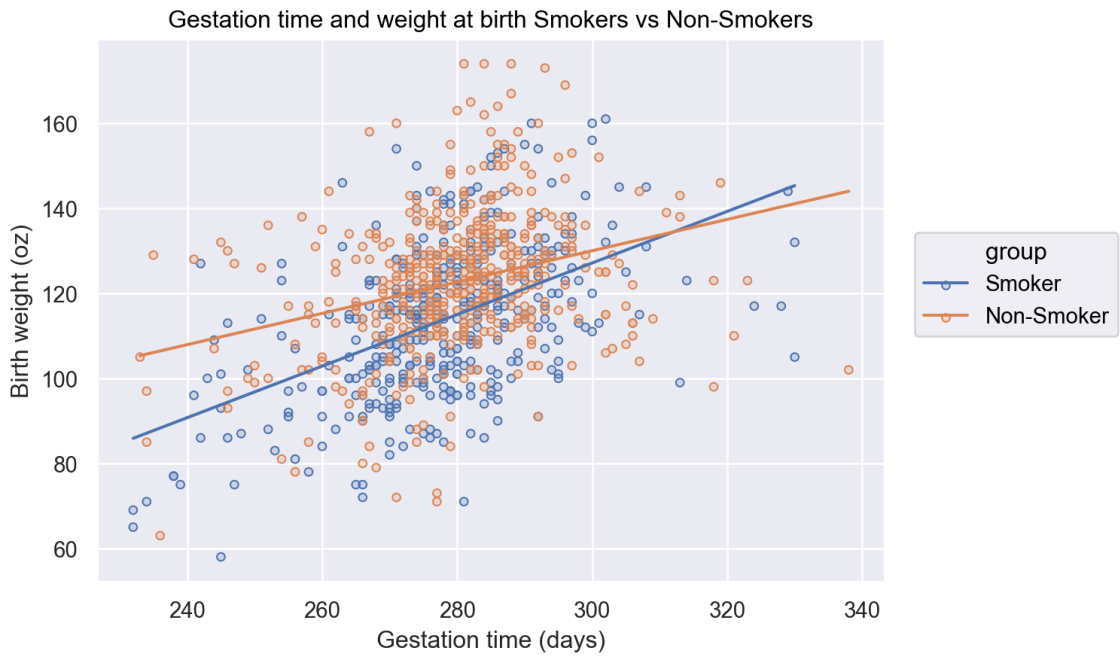
[30]:
```
# for smoking mothers
smokers_bwt = smokers["bwt.oz"]
smokers_gestation = smokers["gestation"]
smokers_data12 = smokers[pd.notnull(smokers_bwt) & pd.
 ↪notnull(smokers_gestation)]
smokers_data12["group"]="Smoker"

# for non_smoking mothers
non_smokers_bwt = non_smokers["bwt.oz"]
non_smokers_gestation = non_smokers["gestation"]
non_smokers_data12 = non_smokers[pd.notnull(non_smokers_bwt) & pd.
 ↪notnull(non_smokers_gestation)]
non_smokers_data12["group"]="Non-Smoker"

# dataframe
plot_data12 = pd.concat([smokers_data12, non_smokers_data12])

# plot
plot12 = so.Plot(plot_data12, x="gestation").add(so.Dots(), y="bwt.oz",␣
 ↪color="group").add(so.Line(),  so.PolyFit(order=1), y="bwt.oz",␣
 ↪color="group").label(title="Gestation time and weight at birth Smokers vs␣
 ↪Non-Smokers", x="Gestation time (days)", y="Birth weight (oz)")
plot12
```

[30]:



[31]:
```
# more than 225 days of gestations
plot_data12_225=plot_data12[plot_data12["gestation"]>225]
plot12_225 = so.Plot(plot_data12_225, x="gestation").add(so.Dots(), y="bwt.oz",␣
↪color="group").add(so.Line(),  so.PolyFit(order=1), y="bwt.oz",␣
↪color="group").label(title="Gestation time and weight at birth Smokers vs␣
↪Non-Smokers", x="Gestation time (days)", y="Birth weight (oz)")
plot12_225
```

[31]:



[32]:
```
results["ex2_birth_weight_lower"] = "yes"
```

1.4 Linear Regression

PLEASE CONTINUE TO SUBSET FOR GESTATION GREATER THAN 225 DAYS for this section

1.4.1 Question 1-3

Now check this relationship using `statsmodels`. Regress birthweight on gestational period and whether the infant's mother smoked. How do you feel about the results? Store the coefficient for smoking in results under the key `"ex3_smoking_coefficient"`.

Note: coefficients are stored in the `.params` attribute of your fit model.

Note 2: you may hit a problem because of the name of one of your columns. You should probably be able to guess the problem given your experience with Python. Also, if you have trouble figuring out how to access the parameters from a fitted model, check out the docs here.

[33]:
```
smoking_gestation_225 = plot_data12

# rename birth weight category without . in it
smoking_gestation_225 = smoking_gestation_225.rename(columns={"bwt.oz":
 ↪"bwt_oz"})

# get coefficient
mod = smf.ols("bwt_oz ~ gestation + smoke", data=smoking_gestation_225).fit()
```

```
results["ex3_smoking_coefficient"] = mod.params["smoke"]
```

1.4.2 Question 1-4

Now let's expand our model to also take into account mothers' pregnancy weight and race (make sure to treat race as a categorical variable!). Don't try to do any recoding of `mrace`, just turn it into a categorical. Does your interpretation of the `smoke` coefficient change? Store the new coefficient under the key `"ex4_smoking_coefficient"`.

[34]:
```
# convert to categorical
smoking_gestation_225["mrace"] = smoking_gestation_225["mrace"].
↪astype("category")

mod = smf.ols("bwt_oz ~ gestation + smoke + mpregwt + mrace",␣
↪data=smoking_gestation_225).fit()
results["ex4_smoking_coefficient"]=mod.params["smoke"]
```

1.4.3 Question 1-5

Now let's test for whether there is an interaction between the mother's race and the effect of smoking.

Note that race is coded as follows:

```
mrace     mother's race or ethnicity
          0-5= white
          6  = mexican
          7 = black
          8 = asian
          9 = mix
          99 = unknown
```

As most variation in this data is between "white" and other categories, first recode race to be an indicator for white and non-white for easier interpretation.

[35]:
```
# white < 6 and non-white > 5, switching back to numeric to get data
smoking_gestation_225["white"] = "non_white"
smoking_gestation_225.loc[smoking_gestation_225["mrace"].astype("float") < 6,␣
↪"white"] = "white"

smoking_gestation_225["white"] = smoking_gestation_225["white"].
↪astype("category")
```

1.4.4 Question 1-6

Now, using post-regression test syntax (not by running a new regression on a subpopulation), recover the coefficient and t-statistic for *whether smoking reduces birth weight for white mothers.* Store the t-statistic of your test in `ex6_compare_white_black_tstat`.

Hint: Read to the BOTTOM of the post-regression test syntax documentation for how to specify hypotheses with strings.

How does this coefficient compare to that for non-white mothers? Is the reduction in birth weight associated with smoking for white mothers larger or smaller than for non-white mothers? Store your answer as either `"larger"` or `"smaller"` under the key `"ex6_compare_white_black"`.

Note: Please ensure your t-stat is stored as a single number, not an array or an array of arrays.

[36]:
```
# model with interaction between smoking and white mothers
mod16 = smf.ols("bwt_oz ~ gestation + smoke + white + smoke:white",␣
 ↪data=smoking_gestation_225).fit()

# t-test for difference between white and non-white smoking effects
hypothesis1 = "smoke:white[T.white] = 0"
t_test = mod16.t_test(hypothesis1)
results["ex6_compare_white_black_tstat"]=t_test.tvalue[0][0]

# coefficient for white mothers
coef_white = mod16.params["smoke"] + mod16.params["smoke:white[T.white]"]
# coefficient for non-white mothers
coef_nonwhite = mod16.params["smoke"]

print(coef_white)
print(coef_nonwhite)

results["ex6_compare_white_black"] = "smaller"
```

```
-8.172214750006809
-9.899620439642073
```

1.4.5 Question 1-7

Now let's use this model to predict some values. Let's generate some hypothetical newborns:

```
newborns = pd.DataFrame(
    {
        "smoke": [1, 1, 0, 0],
        "white": [True, False, True, False],
        "gestation": [253, 300, 248, 287],
        "mpregwt": [132, 129, 140, 139],
    }
)
```

Using the model you ran above with gestation, smoke, mpregwt, white, and the interaction of white and smoke, predict birth weights for these newborns. Store your predictions **in a list** under the key `"ex7_predictions"`.

Note that if you have different data types from those in this pretend dataset or different column names from those in the dataset you used to fit your model, you'll get an error—just adjust the column names / data types to match the data you used to fit your model.

[37]:
```
newborns = pd.DataFrame(
    {
        "smoke": [1, 1, 0, 0],
        "white": ["white", "non_white", "white", "non_white"],
        "gestation": [253, 300, 248, 248],
        "mpregwt": [132, 129, 140, 139],
    }
)

newborns["white"] = newborns["white"].astype("category")
ex7_predictions = [mod16.predict(newborns)]
results["ex7_predictions"]=ex7_predictions
```

1.5 statsmodels versus R

A quick but important note: the tools that are made available in different packages is often a function of who uses those packages, and how they use them. By and large, nearly all statisticians use R, and so many stats tools (like automatic forward-model-selection or backwards-model-selection) have "convenience implementations" (single functions that do all the things you'd want to accomplish) in R, but aren't available as convenience packages in `statsmodels`. That's because `statsmodels` was mostly written by economists and social scientists who tend to feel model selection should be a function of theory not statistical performance (not taking sides: just reporting a difference that exists).

To be clear, you can still implement things like forward model selection yourself in Python – just write a loop that tries different regressors and plots the resulting AICs! – but it will often take a little more work. (Indeed, you can find examples of people writing these loops on the web). And that's not to say that `statsmodels` doesn't have a TON of stuff (take a minute to just look over all the examples of cool things you can do in statsmodels here). Just don't be surprised if you ommissions that seem odd given what you see emphasized in stats courses.

This is one of the reasons that languages are sticky: once a group of people have invested in adding all the bells and whistles they like to a language, there are good reasons to not move to another language, even if the other language has some advantages. A statistician who likes the basic language organization of Python more than R, for example, may still stay with R because the packages already do everything a statistician wants to do, and so it's not worth having to re-implement common tasks in a new language.

1.6 Logistic Regression

1.6.1 Question 1-8

Now, using statsmodels, evaluate the impact of smoking on the likelihood a child is born prematurely (where "premature" is defined as gestation of *less than* 252 days).

For obvious reasons, **DO NOT USE OUR SUBSET FOR GESTATION GREATER THAN 225 DAYS** from above for this section.

For this model, please include mother's pre-pregnancy weight, smoking status, and whether the mother is White (if you use all the racial categories, the model won't converge). Please

don't include the interaction we used in the last question. Store the coefficient for smoking in `ex8_smoking_coefficient`.

[38]:
```
smoking_premature = smoking.copy()
# create premature variable
smoking_premature["premature"] = (smoking_premature["gestation"] < 252).
↪astype(int)

smoking_premature["premature"].value_counts()
```

[38]:
```
premature
0    826
1     43
Name: count, dtype: int64
```

[39]:
```
# add white category with binary variables
smoking_premature["white"]=0
smoking_premature.loc[smoking_premature["mrace"].astype("float") < 6, "white"]␣
↪= 1

mod18 = smf.logit("premature ~ mpregwt + smoke + white",␣
↪data=smoking_premature).fit()
results["ex8_smoking_coefficient"]=mod18.params["smoke"]
print(float(mod18.params["smoke"]))
```

```
Optimization terminated successfully.
         Current function value: 0.187899
         Iterations 7
0.4252773397837855
```

1.7 Weighting

Let's now try using statsmodels on another dataset: drug arrest data from the state of California.

Though California has passed a number of laws lessening penalities for marijuana possession over the years, arguably the biggest changes were in 2010, when the state changed the penalty for possessing a small amount of marijuana from a criminal crime to a "civil" penality (meaning those found guilty only had to pay a fine, not go to jail), though possessing, selling, or producing larger quantities remained illegal. Then in 2016, the state fully legalized marijuana for recreational use, not only making possession of small amounts legal, but also creating a regulatory system for producing marijuana for sale.

Proponents of drug legalization have long argued that the war on drugs contributes to violent crime by creating an opportunity for drug dealers and organized crime to sell and distribute drugs, a business which tends to generate violence when gangs battle over territory. According to this theory, with drug legalization, we should see violent crime decrease after legalization in places where drug arrests had previously been common.

We want to run analysis on crime in California counties to see if, following drug legalization, there was a larger decline in violent crime in counties that had previously also had high drug arrest rates

(to test the idea that violence was being generated by the drug trade, and that legalization would decrease this violence).

In simple analysis, we can treat counties as equally-weighted units of analysis. If we think that each county is a single "community", and we think crime and arrests is shaped at the level of communities (particularly when our community division is related to administrative boundaries that impact policing and government services, and is the case with counties), then this is reasonable.

But one might think that crime is determined at the individual level, or maybe neighborhood level, and so big counties should "count more" in our analysis.

(More formally, when we are fitting a regression model that picks parameter values to minimize the overall sum of squared errors, maybe we should pick values that to a better job of minimizing the errors for populous counties at the expense of not doing quite as good a job of minimizing the errors for smaller counties)

Here, let's use weighted least squares to weight observations based on population. This will allow bigger counties to influence our estimates more.

1.7.1 Question 1-9

To begin, read in our data on California arrest rates and population from Moodle. You will see I've reshaped the data to long format for convenience. This data is stored as a parquet file, so you'll need to install an engine (I usually use `fastparquet` to load the data).

[40]:
```
arrests = pd.read_parquet("arrest_rates_and_population.parquet")
```

[41]:
```
arrests.head()
```

[41]:
```
variable          COUNTY  year  drug_arrest_rate  total_population  \
0         Alameda County  2009          0.003946         1457095.0
1         Alameda County  2018          0.000652         1629615.0
2          Alpine County  2009          0.001735            1153.0
3          Alpine County  2018          0.000831            1203.0
4          Amador County  2009          0.002655           38039.0

variable  violent_arrest_rate
0                    0.002963
1                    0.002536
2                    0.006938
3                    0.004156
4                    0.002629
```

1.7.2 Question 1-10

To run our difference-in-difference in a regression framework, we need both an indicator variable for observations that occur after legalization (i.e. year == 2018), and an indicator for the population we consider treated (those who had drug arrest rates above the average 2009 drug arrest rate). Re-create those here.

As a check, store the number of counties that have a high drug arrest rate in 2009 under the key `"ex10_num_high_drug_arrest"`.

Note: if you want, you can also use the 2009 drug arrest rates as a continuous variable. This is a kind of "generalized difference-in-difference", in which we're just doing the continuous analogue of the thing we do when we split the sample into "high" and "low" 2009 arrest rates.

[42]:
```
# data after legalization
arrests["after_legalization"]=(arrests["year"]==2018).astype(int)

# average 2009 drug arrest rate
mean_2009_arrests = arrests.loc[arrests["year"]==2009, "drug_arrest_rate"].
 ↪mean()

# high drug arrests in 2009
high_drug_2009 = (arrests["drug_arrest_rate"] >= mean_2009_arrests).
 ↪astype("int")
arrests["high_drug_2009"]=high_drug_2009

results["ex10_num_high_drug_arrest"] = high_drug_2009.sum()
print(high_drug_2009.sum())
```

```
24
```

1.7.3 Exercise Question 1-11

Now regression the violent arrest rate on your two indicators and their interaction. The coefficient on the interaction is your "difference-in-difference" estimate! If you go back to your old homeworks, you should find the coefficient is exactly what you calculated. Store this coefficient under the key `"ex11_diff_in_diff"`.

[43]:
```
mod11 = smf.ols("violent_arrest_rate ~ after_legalization + high_drug_2009 +␣
 ↪after_legalization:high_drug_2009", data=arrests).fit()
results["ex11_diff_in_diff"] = mod11.params["after_legalization:high_drug_2009"]
print(float(mod11.params["after_legalization:high_drug_2009"]))
```

```
0.0009664462133596736
```

1.7.4 Question 1-12

Now let's do the same analysis, but this time using county population as a weight in a Weighted Least Squares regression (`smf.wls`). Just pass the keyword argument `weights` as a vector to weights (in this case, `arrests['total_population']`). Note that the `weights` argument is *outside* the formula, so you have to pass an actual vector of values, not just the name of a column.

Does your interpretation of the effects of drug legalization change? Store the new difference-in-difference coefficient under the key `"ex12_diff_in_diff_weighted"`.

[44]:
```

```

```
mod12 = smf.wls("violent_arrest_rate ~ after_legalization + high_drug_2009 +␣
↪after_legalization:
↪high_drug_2009",data=arrests,weights=arrests["total_population"]).fit()
results["ex12_diff_in_diff_weighted"]=mod12.params["after_legalization:
↪high_drug_2009"]
print(float(mod12.params["after_legalization:high_drug_2009"]))
```

```
0.0012023731489897715
```

[45]:
```
# check results
assert set(results.keys()) == {
    "ex2_birth_weight_lower",
    "ex3_smoking_coefficient",
    "ex4_smoking_coefficient",
    "ex6_compare_white_black",
    "ex6_compare_white_black_tstat",
    "ex7_predictions",
    "ex8_smoking_coefficient",
    "ex10_num_high_drug_arrest",
    "ex11_diff_in_diff",
    "ex12_diff_in_diff_weighted",
}
```

1.8 LASSO and Ridge

1.8.1 Question 1-13

Let's go back a little to the `smoking` dataset. You will run a LASSO regression and a ridge regression to penalize the high correlated features. First, we will need to remove the feature `id`. Then, print the correlation matrix and tell me whether there are high correlated features. Why high correlations among features can harm our analysis?

[46]:
```
smoking_no_id = smoking.drop(columns=["id"])

# rename birth weight category
smoking_no_id = smoking_no_id.rename(columns={"bwt.oz":"bwt_oz"})

# correlations
print(smoking_no_id.corr())
```

```
               date  gestation    bwt_oz    parity     mrace      mage  \
date       1.000000   0.064532  0.091575  0.079779  0.055559  0.064616
gestation  0.064532   1.000000  0.394915 -0.132691 -0.145970 -0.030700
bwt_oz     0.091575   0.394915  1.000000  0.041070 -0.130040  0.044344
parity     0.079779  -0.132691  0.041070  1.000000  0.148708  0.523690
mrace      0.055559  -0.145970 -0.130040  0.148708  1.000000  0.013826
mage       0.064616  -0.030700  0.044344  0.523690  0.013826  1.000000
med       -0.066216   0.064221  0.038332 -0.201010 -0.079325  0.134044
mht       -0.066412   0.060726  0.187758 -0.042816 -0.165220 -0.005471
```

```
mpregwt    0.024511   0.068668  0.182116  0.150538  0.022947  0.146137
inc        0.067462   0.027289  0.002173  0.009361 -0.121575  0.296621
smoke     -0.078510  -0.058317 -0.249063  0.011061 -0.113625 -0.069866

                med       mht   mpregwt       inc     smoke
date      -0.066216 -0.066412  0.024511  0.067462 -0.078510
gestation  0.064221  0.060726  0.068668  0.027289 -0.058317
bwt_oz     0.038332  0.187758  0.182116  0.002173 -0.249063
parity    -0.201010 -0.042816  0.150538  0.009361  0.011061
mrace     -0.079325 -0.165220  0.022947 -0.121575 -0.113625
mage       0.134044 -0.005471  0.146137  0.296621 -0.069866
med        1.000000  0.115444 -0.053850  0.216678 -0.137885
mht        0.115444  1.000000  0.460446  0.071477  0.041168
mpregwt   -0.053850  0.460446  1.000000 -0.005130 -0.048551
inc        0.216678  0.071477 -0.005130  1.000000  0.006528
smoke     -0.137885  0.041168 -0.048551  0.006528  1.000000
```

There are no highly correlated features in the matrix, with the highest being around 0.52 and most being much lower. For the correlation of each feature with birth weight, the highest is only around 0.39.

High correlations features can harm the analysis because they can lead to almost singular design matrices, creating large standard errors and poor estimates of coefficients.

1.8.2 Question 1-14

Yes, it is time to run a linear regression, a LASSO regression and a ridge regression to predict the birth weight `bwt.oz`. You will need to search the best parameters in your LASSO and ridge fittings. It is useful to plot the changes of the estimations of the coefficients of your models with respect to different penalty parameters. Give interpretation and comparations among the three fittings.

[47]:
```
# standardize and split data
X = smoking_no_id.drop(columns=["bwt_oz"])
y = smoking_no_id["bwt_oz"]

columns = X.columns

X = preprocessing.scale(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=10)

# linear regression
ols_reg = LinearRegression()
ols_reg.fit(X_train, y_train)
ols_pred = ols_reg.predict(X_test)

coef_ols = pd.DataFrame({'variable': columns, 'estimate': ols_reg.coef_})
coef_ols
```

[47]:

	variable	estimate
0	date	1.000329
1	gestation	6.634415
2	parity	1.744228
3	mrace	-1.475781
4	mage	-0.291379
5	med	-0.058792
6	mht	1.784272
7	mpregwt	2.550383
8	inc	-0.850810
9	smoke	-4.638054

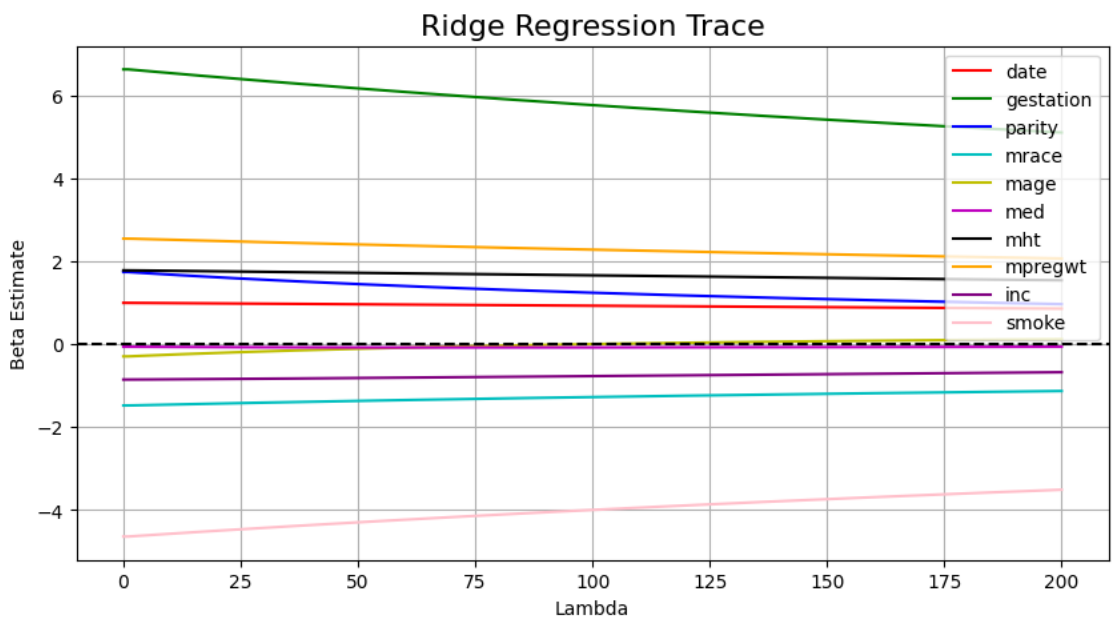
With the linear regression between birth weight and the other features, gestation has the strongest coefficient which is positive, and pregnancy weight, height, and parity are also positive. Smoking has the strongest negative coefficient with race also having a weaker negative coefficient.

[48]:
```
columns
```

[48]:
```
Index(['date', 'gestation', 'parity', 'mrace', 'mage', 'med', 'mht', 'mpregwt',
       'inc', 'smoke'],
      dtype='object')
```

[49]:
```
# for warnings
warnings.simplefilter(action="ignore", category=pd.errors.PerformanceWarning)


# ridge regression
ridge_reg = Ridge(alpha=0)
ridge_reg.fit(X_train, y_train)
ridge_df = pd.DataFrame({'variable': columns, 'estimate': ridge_reg.coef_})
ridge_train_pred = []
ridge_test_pred = []

alphas = np.arange(0, 200, 1)

for alpha in alphas:
    ridge_reg = Ridge(alpha=alpha)
    ridge_reg.fit(X_train, y_train)
    var_name = 'estimate' + str(alpha)
    ridge_df[var_name] = ridge_reg.coef_

    ridge_train_pred.append(ridge_reg.predict(X_train))
    ridge_test_pred.append(ridge_reg.predict(X_test))

ridge_df = ridge_df.set_index('variable').T.reset_index(drop=True)

fig, ax = plt.subplots(figsize=(10,5))
```

```
ax.plot(ridge_df.date, 'r', ridge_df.gestation, 'g', ridge_df.parity, 'b',␣
↪ridge_df.mrace, 'c', ridge_df.mage, 'y', ridge_df.med, 'm', ridge_df.mht,␣
↪'k', ridge_df.mpregwt, 'orange', ridge_df.inc, 'purple', ridge_df.smoke,␣
↪'pink')
ax.axhline(y=0, color='black', linestyle='--')
ax.set_xlabel("Lambda")
ax.set_ylabel("Beta Estimate")
ax.set_title("Ridge Regression Trace", fontsize=16)
ax.legend(labels=ridge_df.columns, loc='upper right')
ax.grid(True)
```

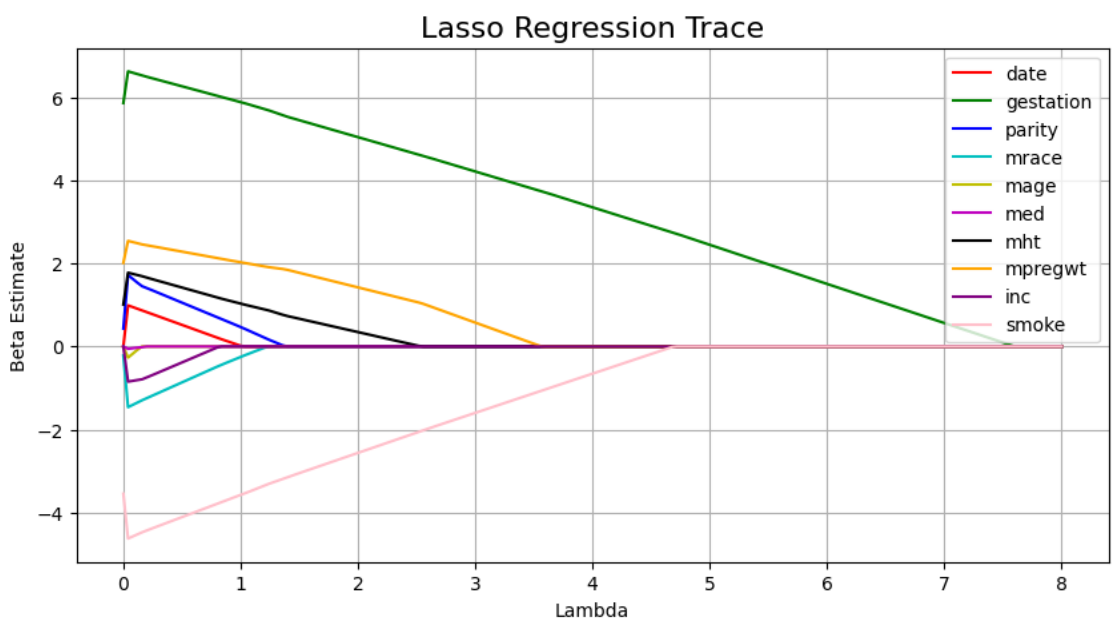


The same effects can be viewed here with gestation having the strongest positive coefficient, and pregnancy weight, height, parity, and date showing positive lines converging to 0. Gestation and parity do seem to converge slightly faster than the other positive features. The strongest negative coefficient is smoking which an be viewed here with a pretty clear convergence to 0 that seems to match the other negative lines in this plot.

[50]:
```
# lasso regression
lasso_reg = Lasso(alpha=1)
lasso_reg.fit(X_train, y_train)
lasso_df = pd.DataFrame({'variable': columns, 'estimate': lasso_reg.coef_})
lasso_train_pred = []
lasso_test_pred = []

alphas = np.arange(0.01, 8.01, 0.04)

for alpha in alphas:
```

```
    lasso_reg = Lasso(alpha=alpha)
    lasso_reg.fit(X_train, y_train)
    var_name = 'estimate' + str(alpha)
    lasso_df[var_name] = lasso_reg.coef_

    lasso_train_pred.append(lasso_reg.predict(X_train))
    lasso_test_pred.append(lasso_reg.predict(X_test))

lasso_df = lasso_df.set_index('variable').T.reset_index(drop=True)

fig, ax = plt.subplots(figsize=(10,5))
ax.plot(lasso_df.date, 'r', lasso_df.gestation, 'g', lasso_df.parity, 'b',␣
 ↪lasso_df.mrace, 'c', lasso_df.mage, 'y', lasso_df.med, 'm', lasso_df.mht,␣
 ↪'k', lasso_df.mpregwt, 'orange', lasso_df.inc, 'purple', lasso_df.smoke,␣
 ↪'pink')
ax.set_xlabel("Lambda")
ax.set_ylabel("Beta Estimate")
ax.set_xticklabels(np.arange(-1,10,1))
ax.set_title("Lasso Regression Trace", fontsize=16)
ax.legend(labels=ridge_df.columns, loc='upper right')
ax.grid(True)
```

```
C:\Users\tessa\AppData\Local\Temp\ipykernel_26440\4053382427.py:25: UserWarning:
set_ticklabels() should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
  ax.set_xticklabels(np.arange(-1,10,1))
```



With the lasso regression, gestation period shows the longest line before converging to zero, marking it as a more important feature (with a strong positive coefficient as shown before). The most negative influence comes from smoking which also converges to 0 but with a longer line than most features. The features with less effect such as age, education and income (shown before with coefficients near 0 and ridge regression lines close to the x-axis) converge to 0 with much smaller lines.